

20250912

大概是后面你俩可能接触到的东西

stm32从hal库('HAL_')开始学吧, hal更现代化 标准库 ('GPIO.') 更底层, 需要掌握的概念更多, 更容易出错, 开发周期 (点灯, 类似小项目的全流程完成时间) 更长; 【现行市面上的代码, hal占比更多, 标准库比较少】

下面是, 以机器人入门, 分阶段展开的所需要学习的内容

有你俩学的stm32, 大概在276行, “阶段一”

一点建议:

善用ai工具, 比如chatgpt,deepseek
掌握电脑翻墙 (vpn) 方案

明白! 我帮你做一份 **针对你的四层架构** 的学习计划, 既考虑到你要用到的技术, 又考虑到时间紧迫 (你说几个月后要用上)。我会按 **阶段目标 + 学习内容 + 实践任务** 来安排, 让你逐步上手。

学习计划: 从 0 到能跑你的机器人架构

阶段 1: C 语言 + STM32 裸机入门 (2~3 周)

目标: 会写简单的 MCU 驱动, 能点灯、读传感器、发串口。 **学习内容:**

- C 语言: 数据类型、指针、数组、函数、结构体
- STM32 基础: CubeMX、HAL 库、GPIO、UART、PWM、ADC
- 常见外设: LED、按键、串口、定时器中断 **实践任务:**
- 用 CubeMX 生成最小工程, 点亮 LED
- 用 UART 打印字符串到串口助手
- 用定时器中断周期性翻转 LED
- 读一个简单传感器 (霍尔、按键) 并串口输出

阶段 2: FreeRTOS 入门 (2~3 周)

目标: 把裸机代码改造成多任务, 能跑并行任务, 学会用队列/信号量。 **学习内容:**

- 任务 (Task) 创建、优先级、堆栈大小
- 调度器、时间片、中断安全
- 队列 (Queue)、二值信号量、互斥锁
- 软件定时器 **实践任务:**
- 创建两个任务: LED 闪烁 + 串口打印

- 用队列把 UART 接收数据传给另一个任务处理
 - 模拟电机控制：高优先级任务实时输出 PWM
 - 验证优先级抢占、任务延时、死锁调试
-

🌀 阶段 3：C++ 基础 + 任务封装（2 周）

目标：学会用类和对象写任务，让代码结构清晰，便于扩展。 **学习内容：**

- C++ 基础：类、对象、构造函数、继承、引用、智能指针
 - 用 C++ 封装 FreeRTOS 任务（把任务函数放到类成员）
 - 学会用模板和命名空间组织代码 **实践任务：**
 - 把你的电机控制任务封装成类，支持多电机对象
 - 用类封装 CAN 通信模块，支持多节点数据收发
-

🌀 阶段 4：Python 基础 + 上位机调试（2 周）

目标：能写脚本和可视化工具，调试下位机。 **学习内容：**

- Python 语法：列表、字典、函数、类
 - 串口通信：pyserial
 - 可视化：matplotlib、numpy **实践任务：**
 - 写一个 Python 脚本通过串口发控制命令给 STM32
 - 实时画出下位机回传的传感器数据曲线
 - 用脚本调试电机转速、PID 参数
-

🌀 阶段 5：ROS 节点开发 + 系统集成（3~4 周）

目标：把你的树莓派和主控连通，能跑完整系统。 **学习内容：**

- ROS 1 Noetic 基础：话题、节点、消息、rviz
 - ROS C++ (roscpp) 和 Python (rospy) 节点编写
 - ROS 上位机订阅底盘里程计、发布速度指令
 - micro-ROS：在 STM32 上运行 ROS 节点 **实践任务：**
 - 树莓派跑 ROS master，订阅 STM32 里程计
 - 树莓派发布速度指令，STM32 收到后控制电机
 - 用 rviz 可视化机器人轨迹
 - 预留接口给深度相机、雷达（先用仿真数据）
-

🔲 阶段 6：整合优化 + 项目调试（持续）

- 路径规划：实现 RRT 或 A* 规划器（C++/Python）
 - 数据融合：用 ROS 的 robot_localization 融合 IMU + 里程计
 - 性能优化：给控制任务最高优先级，保证实时性
 - 安全策略：CAN 容错、急停逻辑
-

🔗 学习顺序建议

1. 先搞定 **STM32 裸机 + FreeRTOS** (核心!)
2. 再学 Python (调试和上位机)
3. 同步学 ROS 节点开发 (最好 C++ + Python 都能看懂)
4. 最后整合系统, 做实机调试

要不要我帮你做一张 **学习路线图 (可视化流程图)**, 从 C → FreeRTOS → C++ → Python → ROS, 直观显示每一步目标和输出成果? 这样你能打印出来贴在桌子上, 跟着一步一步做。

明白! 既然你要更详细, 我给你把整个学习计划**完全展开**, 做到每一步都有学习目标、知识点、具体实践任务、参考资料, 形成一条清晰的执行路线, 你可以每天按步骤推进。

🌀 机器人多层架构学习计划 (超详细版)

🌀 阶段 1: C 语言 + STM32 裸机入门 (2~3 周)

🌀 阶段目标

- 熟悉 C 语言基本语法、指针和结构体
- 会用 CubeMX 生成工程, 使用 HAL 库控制外设
- 能点灯、读传感器、用串口收发数据

📖 学习内容

- **C 语言核心**
 - 基本语法: 变量、运算符、条件、循环
 - 数组、指针、字符串、结构体
 - 函数调用、传参、作用域
- **STM32 基础**
 - CubeMX 配置 GPIO、UART、TIM
 - HAL 库函数使用 (HAL_GPIO_TogglePin、HAL_UART_Transmit 等)
- **外设驱动**
 - LED 点亮、按键消抖
 - UART 收发、定时器中断

🌀 实践任务

1. 最小工程

- 用 CubeMX 生成工程, 配置系统时钟
- 点亮 LED (用 HAL_GPIO_WritePin)

2. 串口调试

- 用 UART 打印 "Hello World"
- 用串口助手发送数据，MCU 回显

3. 定时器中断

- TIM 中断周期翻转 LED
- 改变定时器周期，观察闪烁速度变化

4. 传感器输入

- 接一个按键，按下串口输出 "Pressed"
- 学会按键消抖

📖 推荐资料

- 《C语言程序设计（谭浩强）》或《C Primer Plus》
- STM32CubeMX 官方文档
- HAL 库参考手册
- B站搜索「STM32CubeMX 入门」「HAL库教程」

🎯 阶段 2：FreeRTOS 入门（2~3 周）

🎯 阶段目标

- 学会创建多任务、切换、通信
- 会用队列、信号量解决任务间通信问题
- 能把裸机工程改造成多任务工程

📖 学习内容

- **FreeRTOS 核心**
 - 任务创建 (`xTaskCreate`)、删除、延时 (`vTaskDelay`)
 - 调度器原理、优先级抢占
 - 队列 (Queue) 发送/接收
 - 二值信号量、互斥锁
- **中断与 RTOS**
 - 中断安全函数 (FromISR 系列)
 - 任务通知机制

🔗 实践任务

1. 多任务创建

- Task1: 500ms 闪灯
- Task2: 1s 打印 "Hello"
- 观察两任务交替运行

2. 任务通信

- UART 中断接收 → 用队列传给 Task2 → Task2 打印

3. 任务优先级实验

- 创建一个高优先级任务快速闪灯，验证优先级抢占

4. 互斥锁实验

- 两个任务竞争访问串口，避免输出打架

📖 推荐资料

- 《Mastering FreeRTOS》电子书
- FreeRTOS 官方网站文档
- B站搜索「FreeRTOS 入门教程」「任务、队列、信号量」

🌀 阶段 3：C++ 基础 + 封装任务（2 周）

🎯 阶段目标

- 掌握 C++ 面向对象编程
- 会用类封装任务、驱动，代码结构更清晰

📖 学习内容

- 类与对象、构造函数/析构函数
- 引用、指针、命名空间
- 模板、继承、虚函数
- RAII、内存管理、智能指针

🌀 实践任务

1. 用类封装任务

- 创建一个 LED 控制类，成员函数作为任务入口

2. 多对象控制

- 用两个对象控制两路 LED，互不干扰

3. 封装驱动

- 把 UART 驱动写成一个类，支持多实例

🌀 阶段 4：Python 基础 + 上位机调试（2 周）

🎯 阶段目标

- 会用 Python 写调试脚本

- 能实时画出下位机回传数据

📖 学习内容

- Python 基本语法：变量、列表、字典、类
- 串口通信：pyserial
- 数据可视化：matplotlib、numpy

🌀 实践任务

1. 串口通信脚本

- 发送控制命令给 STM32，让 LED 亮/灭
- 接收回传数据打印在终端 2

仅展开阶段一

好的，我们就把 **阶段一：C语言 + STM32 裸机入门** 完整展开，做到每天都能跟着学和练。这样你能真正把 MCU 基础打牢。

🌀 阶段 1：C 语言 + STM32 裸机入门（2~3 周）

目标：

- 掌握 C 语言基本语法、指针、结构体
- 会用 CubeMX 生成工程、用 HAL 库驱动外设
- 能点亮 LED、用串口收发数据、读传感器、用定时器中断

📅 建议学习时间表（15 天）

天数	学习重点	具体任务	输出成果
Day 1	C 语言基础	变量、数据类型、运算符、控制语句 (if/for/while)	用 PC 打印 1~10 的和
Day 2	函数与作用域	写函数求和、求最大值，理解局部/全局变量	函数调用，打印结果
Day 3	数组与字符串	学会数组、字符串操作	写程序反转字符串
Day 4	指针入门	理解地址、指针、数组与指针关系	用指针操作数组
Day 5	结构体与枚举	用结构体存储多变量，用枚举表示状态机	打印结构体内容
Day 6	STM32 基础概念	学习 Cortex-M3 架构、CubeMX 工程生成流程	生成第一个空工程
Day 7	GPIO 控制	配置 LED 引脚，使用 HAL_GPIO_TogglePin	LED 闪烁 1Hz
Day 8	串口通信	配置 UART1，用 HAL_UART_Transmit 打印“Hello”	串口助手能接收到

天数	学习重点	具体任务	输出成果
Day 9	UART 接收	学习 HAL_UART_Receive_IT	PC 发送字符，MCU 回显
Day 10	定时器中断	配置 TIM3 1kHz 中断，回调中翻转 LED	LED 稳定闪烁
Day 11	按键输入	配置按键 GPIO + 外部中断	按下按键 → 串口输出
Day 12	消抖处理	软件消抖，延时判断按键稳定	串口输出只触发一次
Day 13	模拟传感器输入	使用 ADC 读取电位器电压	串口输出电压值
Day 14	综合练习	用串口命令控制 LED、读按键、读电压	完成简单人机交互
Day 15	总结与整理	整理代码结构、写笔记、画流程图	输出一份“阶段一笔记”

学习内容细分

1 C 语言核心

- **语法**：变量、常量、运算符、条件判断、循环
- **数组和字符串**：char 数组、字符串结束符 `\0`
- **指针**：指针变量、指针运算、函数参数传递
- **结构体**：封装多变量，简化传参
- **函数**：值传递、地址传递

推荐资料：

- 《C Primer Plus》前 8 章（指针部分重点看）
- 在线 C 语言练习平台（菜鸟教程、LeetCode 简单题）

2 STM32 + CubeMX 入门

- CubeMX 工程生成、时钟配置
- GPIO 模式：推挽输出、上拉输入
- UART 配置：波特率、收发中断
- TIM 配置：定时器溢出中断、回调函数

推荐资料：

- STM32CubeMX 官方文档
- HAL Driver User Manual
- B站关键词：“STM32CubeMX GPIO UART 入门”

③ 必做实践任务（里程碑）

☑ 任务 1：点亮 LED

- 用 HAL_GPIO_WritePin 点亮
- 用 HAL_GPIO_TogglePin 周期翻转

☑ 任务 2：串口通信

- 串口助手能收发数据
- 用 printf 重定向到 UART 实现调试输出

☑ 任务 3：定时器中断

- 周期性翻转 LED
- 修改定时器周期，观察闪烁变化

☑ 任务 4：按键输入 + 消抖

- 外部中断触发
- 通过消抖避免抖动误触发

☑ 任务 5：模拟传感器读取

- 使用 ADC 读取电位器
- 用串口打印实时电压值

☑ 任务 6：综合人机交互

- 串口输入 "ON" → LED 亮
- 串口输入 "OFF" → LED 灭
- 输入 "V" → 打印当前电压

🎯 阶段完成标准

- 能独立用 CubeMX 生成一个工程
- 会写 GPIO、UART、TIM、EXTI、ADC 的基本代码
- 能把传感器数据通过串口回传
- 代码结构清晰，能复用

要不要我帮你画一张 **阶段一学习流程图**（包含从 C 学习到点灯、串口、定时器、传感器的路径），这样你一看就知道每天的目标和产出？

更详细、更系统

明白，这里我给你做一个 **超详细、系统化的阶段一学习计划**，专门针对你的目标（STM32 + HAL 裸机开发），让你能稳扎稳打完成“上位机/下位机分层”中的下位机开发入门。

🌀 阶段一：C 语言 + STM32 裸机 + HAL 入门计划 (4~6 周)

🎯 **目标：**能独立用 C 语言 + HAL 库写出一个完整的嵌入式程序，点灯、串口通信、定时器、外部中断、PWM、ADC、编码器、CAN 基本通信都能跑通。🧠 **前提：**完全零基础或仅有一点 C 语言语法概念。

📅 学习节奏建议

- **总时长：**4~6 周，每周 5~6 天，每天 1.5~2 小时。
- **顺序：**先语言 → 再单片机基础 → 再外设 → 最后整合。
- **方式：**先做实验、再理解原理，理论和实践交替。

① C 语言基础（第 1 周）

学习重点

模块	关键知识点	目标
数据类型	int、float、char、unsigned、typedef	会区分有符号/无符号、大小
运算符	算术、关系、逻辑、位运算、优先级	会用 `&` `^` `<<` `>>` 操作寄存器
流程控制	if/else、switch、for、while、break/continue	会写简单状态机
函数	形参/实参、返回值、作用域	能把逻辑拆成函数
数组/指针	一维数组、字符串、指针、地址	会用指针访问数组元素
结构体/枚举	struct、enum、typedef	能封装外设配置参数
存储类型	static、volatile、const	知道中断变量要加 volatile
内存布局	栈/堆/全局变量	知道局部变量作用域

推荐资源

- 书籍：《C语言程序设计（谭浩强）》入门
- 视频：B站 [小甲鱼C语言入门](#)
- 实践：用 Keil 或 VSCode + gcc 写几个小程序
 - 例子：实现一个简易计算器、数组排序、位运算灯光控制模拟

② STM32 基础 + 环境搭建（第 2 周）

学习重点

模块	关键知识点	目标
MCU 基础	GPIO、时钟树、Flash、SRAM	了解芯片结构
HAL 库	初始化、函数命名规则、CubeMX	会生成 HAL 工程
开发环境	CubeMX、Keil、ST-LINK 驱动	能完成工程编译、下载、调试

实践任务

- 用 CubeMX 生成一个点灯工程，下载到开发板。
- 用 HAL_GPIO_TogglePin() 控制 LED 闪烁。
- 熟悉 main.c 的结构、HAL_Init()、SystemClock_Config()。

③ GPIO / 外设驱动入门（第 3 周）

外设	学习目标	实践例程
GPIO	输入、输出、上拉下拉	按键控制 LED
EXTI	外部中断、回调函数	按键中断点灯
TIM	基本定时器、中断、延时	定时 1s 翻转 LED
PWM	占空比控制	调光、直流电机速度控制
USART	串口中断收发、printf 重定向	上位机收发数据
DMA	串口 DMA 发送	连续输出数据不卡顿

💡 建议先 GPIO/EXTI/TIM → 再 USART → 再 PWM/DMA，循序渐进。

④ 进阶外设（第 4~5 周）

外设	学习目标	实践例程
ADC	读取电压、电位器	采集并串口打印值
I2C/SPI	外设通信	驱动一个 MPU6050 或 OLED
Encoder Mode	定时器编码器接口	读取电机位置
CAN	基本帧收发	两块板子互发数据

⑤ 项目整合（第 6 周）

把所有外设组合成一个小项目，比如：

项目建议：小车底盘控制 Demo

- PWM 驱动直流电机
- 编码器反馈速度
- 串口接收目标速度
- 串口回传实际速度
- 用定时器实现 1kHz 控制循环

这样就形成了一个最小闭环，后面迁移到 FreeRTOS 很自然。

学习策略

- **每天都烧一次程序** → 形成肌肉记忆
 - **多看 HAL 源码** → 熟悉寄存器级逻辑
 - **画流程图** → 把中断/任务调度画出来
 - **调试习惯**：会用断点、观察变量、串口调试
-

要不要我帮你画一张 **阶段一学习路线图（时间轴+模块+目标）**，方便你每天按计划执行？这样你可以直接照着走，不会乱掉。